

Midnight: A Philosophy of Minimal Syntactic Ceremony

Core Principle

A language should require only the syntax that carries real meaning, and avoid syntax that exists mostly because of historical design decisions or language limitations.

The goal isn't "use fewer keywords." The goal is:

| **Every keyword should earn its place.**

Three guiding questions: - Does this keyword add meaning? - Can a beginner understand it after reading it once? - Does removing it reduce the language's capabilities?

This aims to eliminate *accidental complexity* (introduced by the language itself) while preserving *essential complexity* (the inherent difficulty of systems programming, which no language can remove).

Important distinction: simple syntax $\neq$ simple language. Python has minimal syntax but deep hidden complexity (decorators, metaclasses, the object model). C has small syntax but is hard to use correctly (manual memory management). The target is syntax that *earns its place*, not syntax that is merely short.

1. Too Much Ceremony — Hello World

Java forces a beginner to learn `public`, `class`, `static`, `void`, `String[]` args — none of which matter for a first program.

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Go strips most of this away:

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, World!")
}
```

Zig keeps some ceremony, but most of it has a purpose:

```
const std = @import("std");

pub fn main() !void {
    std.debug.print("Hello\n", .{});
}
```

Midnight removes it entirely for the first program:

```
print("Hello, World!");
```

Advanced concepts like modules and entry points come later, once the beginner is ready.

2. Struct Declaration

C requires writing struct twice — historical baggage:

```
struct Person {
    char* name;
};

struct Person person;
```

Midnight:

```
struct Person {
    var string name;
}
```

```
var Person person;
```

Same expressive power, no redundant repetition.

3. Function Signatures — Essential vs. Accidental Complexity

C++ often buries a simple idea under layers of syntax:

```
template<typename T>
constexpr inline auto add(const T& a, const T& b) noexcept -> T {
    return a + b;
}
```

The actual idea is: *“a function that adds two values of the same type.”*
Everything else is implementation detail:

Keyword	Purpose	Essential to the idea?
template<typename T>	Generic function	Yes
constexpr	Compile-time execution	Sometimes, not for beginners
inline	Optimization hint	Mostly historical; compilers decide automatically
auto	Return type deduction	Useful
const T&	Pass by const reference	Important for performance, but not for the idea itself
noexcept	No-throw guarantee	Important for optimization, not for the idea
-> T	Trailing return type	Pure syntax

Progressive disclosure in Midnight

Level 1 — beginner:

```
fn add(a: int, b: int) -> int {
```

```
    return a + b;
}
```

Level 2 — generic:

```
fn add<T>(a: T, b: T) -> T {
    return a + b;
}
```

Level 3 — expert, full control:

```
@inline
comptime fn add<T>(
    borrow a: T,
    borrow b: T
) -> T {
    return a + b;
}
```

The language doesn't get more complicated — the programmer simply learns more features as they need them.

Specific replacements

- **constexpr** → `comptime fn` or `const fn` — reads like what it does.
 - **noexcept** → `never throws` or `safe fn` — reads like English.
 - **inline** → removed from the core grammar; LLVM already knows when to inline better than most humans. If truly needed: `@inline` as an attribute, separate from language grammar.
-

4. Making Performance Visible Without Forcing It

Rather than silently copying a large struct or requiring beginners to always write `borrow`, the compiler can *teach*:

```
fn process(person: Person) {
    ...
}
```

If `Person` is large (say, 5 MB), the compiler responds:

`Person` is approximately 5 MB. Passing by value will copy it.
Consider ``borrow person: Person`` if you intended to avoid the copy.

Simple syntax stays simple for newcomers, but when a performance-relevant choice appears, the compiler explains what's happening and how to take control.

5. Memory Allocation

C requires understanding pointers, the heap, `sizeof`, `malloc`, and `free` just to create an object:

```
Person* person = malloc(sizeof(Person));
```

Java hides everything, including where the object lives, when it dies, and who frees it:

```
Person person = new Person();
```

Midnight separates *intent* from *implementation*:

```
var Person person;      // "I want a Person" – compiler decides
                          stack vs heap
```

```
heap Person person;     // explicit heap allocation
var Person* person = new Person{};
```

Beginners never touch heap semantics until they explicitly ask for them.

6. Pointers

C's `int* ptr`; gives no indication of what the `*` means — beginners just have to memorize it.

Midnight names the concept directly:

```
pointer<int> number;
```

The syntax says exactly what it is, and can be visualized directly (address → value in memory).

7. References

C++'s `&` is overloaded with three different meanings: address-of, reference, and bitwise AND.

Midnight:

```
borrow person: Person
```

Immediately understandable — you aren't owning it, you're borrowing it (terminology proven useful by Rust).

8. Ownership

Rust enforces move semantics immediately and rejects use-after-move at compile time — safe, but a harsh first-week experience.

Midnight's proposed approach: teach progressively.

- **Week 1:** compiler allows the pattern.
- **Week 20:** compiler explains:

```
This value is being moved.
Ownership prevents double-free.
Learn more →
```

Instead of a cryptic error E0382, the compiler *teaches* rather than *punishes*.

9. Null Pointers

C (`Person* p = NULL;`) and Java (`Person p = null;`) both allow silent crashes later.

Midnight:

```
optional<Person> person;
// or
```

Person?

The compiler forces a check before use:

```
if person {  
    ...  
}
```

Much harder to accidentally crash.

10. Arrays

C arrays carry no length information, so `numbers[100]` is undefined behavior if the array only has 10 elements.

Midnight:

```
var [10]int numbers;
```

The compiler knows the length and can catch mistakes directly:

Index 100 exceeds array size 10.

11. Error Handling

- C: `if (fopen(...) == NULL)`
- Java: `try { } catch (...) { }`
- Rust: `Result<T, E>`

Midnight aims for simplicity up front, with room to learn the mechanism later:

```
var file = open("hello.txt"?);
```

The `?` is simple to use immediately; what it expands into is a lesson for later.

12. Modules

C's split between `foo.h` and `foo.c` exists for historical reasons.

Midnight:

```
// math.mid  
export fn add(...)
```

```
// other file  
import math;
```

One file, one declaration, one definition.

13. Const Placement

C++'s `const int* const ptr;` trips up even experienced programmers.

Midnight keeps the modifier next to what it modifies:

```
const number = 5  
borrow const person
```

14. Function Declarations

C requires a separate declaration and definition:

```
int add(int,int);      // declaration

int add(int a,int b){  // definition
}
```

Midnight collapses this into one:

```
export fn add(...)
```

15. Struct Initialization

C requires designated-initializer syntax that exists mostly because C had to evolve around its existing grammar:

```
struct Person person = {
    .first_name = "Naveen",
    .last_name = "Hettiwaththa"
};
```

Midnight mirrors the data directly:

```
var Person person = Person{
    first_name = "Naveen",
    last_name = "Hettiwaththa"
};
```

A new language doesn't carry C's historical baggage, so it doesn't need to repeat this pattern.

16. Compiler Diagnostics as a Teaching Tool

This may be Midnight's biggest opportunity. Instead of silently copying a large struct or requiring beginners to know about borrowing from day one, the compiler can offer guided choices:

```
fn process(person: Person) {
}
```

If Person is 18 MB (image + audio + metadata):

Passing 'Person' by value copies approximately 18 MB.

Did you mean one of these?

- ✓ borrow person: Person
No copy.
- ✓ move person: Person
Transfer ownership.
- ✓ copy person: Person
Keep an explicit copy.

The compiler doesn't just reject the code — it explains *why* the performance matters and offers the appropriate tools, teaching systems programming as the user grows into it.

Overall Philosophy: Make the Computer's Behavior Observable

Midnight shouldn't try to hide how computers work — it should make that behavior *visible* rather than either forcing it on beginners or hiding it entirely.

For example, an IDE could show on hover:

Stack-allocated variable:

Variable: person

Storage: ✓ Stack
Size: 64 bytes
Alignment: 8 bytes
Lifetime: Until function returns
LLVM: %person = alloca %struct.Person

Heap-allocated variable:

Storage: ✓ Heap
Allocation: new Person
Owner: main()
Will be freed: When owner releases it

This preserves Midnight as a true systems language — memory, layout, ownership, and performance remain fully reasoned about — but these concepts become observable rather than hidden behind syntax or left for the user to infer through trial and error.

Summary Table

Concept	Problem in existing languages	Midnight's approach
Hello World	Ceremony unrelated to the task	<code>print("Hello, World!");</code>
Structs	Redundant <code>struct</code> keyword	Declared once, used directly
Generics/functions	Buried under templates, <code>noexcept</code> , <code>inline</code>	Progressive disclosure across 3 levels
Memory allocation	Manual (C) or fully hidden (Java)	Intent-based (<code>var</code> , <code>heap</code>) with compiler inference
Pointers	Unexplained <code>*</code>	<code>pointer<T></code>
References	Overloaded <code>&</code>	<code>borrow</code>
Ownership	Immediate strict enforcement (Rust)	Gradual, explanatory enforcement
Null	Silent crash potential	<code>optional<T> / T?</code> forces checks
Arrays	No bounds awareness	Compiler-tracked length, clear errors
	Verbose (Java) or	<code>? operator</code> , learn

Error handling	unfamiliar (Rust)	internals later
Modules	Split header/source files	Single file, export/import
Const	Confusing placement	Modifier stays next to what it modifies
Function declarations	Duplicated in C	Single declaration + definition
Struct initialization	Designated initializer syntax	Mirrors the data directly
Diagnostics	Reject-only errors	Teach-first, suggest concrete fixes
